2D GAMES · SAMPLE BUILD PLAN

Game 01 — Tap or Don't Tap (Complete Plan)

Game 1 of 7 Complexity: ⭐ Low Estimate: 1–2 days Status: not started

> Complete plan (supersedes the 2026-09-09 sample). Status: **built & browser-verified** (2026-09-09 build pass) — `apps/frontend/public/games/tap-or-dont-tap/` with decoys, Stroop traps, rule swap, audio, pause and a baked percentile table. Logic tests: `apps/frontend/src/__tests__/games-tap-or-dont-tap.test.ts`.
> This doc is the spec of record: the constants below are authoritative when tuning, and the open items in §9–§12 are the only remaining work.

1. Overview

A Go/No-Go reaction game (the real psychology self-control test). The shareable number is the **best reaction time in ms**. Success metric: median first session ≥ 15 rounds; players can state their ms score from memory after one session.

2. Complete game spec

Round loop (state machine: `MENU → WAITING → SIGNAL → FEEDBACK → (WAITING | GAMEOVER)`, plus `PAUSED`):

1. **WAITING** — dark screen, random delay **uniform 300–3000 ms**. A tap here is a **false start**: lose a heart, skip the round (no signal shown).
2. **SIGNAL** — one of:
   - 🟢 GREEN → tap within the window.
   - 🔴 RED → do not tap; survives after the window expires.
   - 🟡/🔵 DECOY (round ≥ 10) → never tap; ignoring it for the full window = +50.
   - STROOP TRAP (round ≥ 15) — word "TAP" in red or "WAIT" in green; **color wins, word ignored**.
3. **FEEDBACK** — 400 ms overlay ("218ms! 🔥" / "❌ Too slow" / "✅ Resisted +25"), then 250 ms blank before the next WAITING.

Constants (authoritative — implemented in `game.js`)

Constant	Value	Notes
`WAIT_MIN/MAX_MS`	300 / 3000	uniform random
`WINDOW_START_MS`	1200	signal visible duration, round 1
`WINDOW_SHRINK_MS`	25	per round
`WINDOW_FLOOR_MS`	450	never below
`HEARTS`	3	0 → game over
`GREEN_BASE_POINTS`	100	`points = min(100 + streak×10, 500)` (×5 cap)

Constant	Value	Notes
`RESIST_BONUS`	25	red expired untouched
`DECOY_BONUS`	50	decoy expired untouched (round ≥ 10)
`FEEDBACK_MS`	400	+ 250 ms blank
`SWAP_ROUNDS`	21–23	"🔄 RULES SWAPPED!" — red taps, green doesn't

Signal mix per round band: 1–9 → 45 % green / 55 % red; 10–14 → 40/30/30 decoy;
15+ → 40 green / 30 red / 15 decoy / 15 Stroop.

Losing: tap on red/decoy/Stroop-wrong, false start, or green missed → −1 heart each.
Reaction recorded only on green hits. Swap inverts green/red only; decoys always
"never tap".

3. Progression & percentile model

- Difficulty = shrinking window + signal mix. No other knobs.
- **Best reaction** = min green-hit reaction of the run; **score** = sum of points.
- **Top-% badge**: `BAKED_TABLE` in `game.js` maps score → percentile (fixed simulated distribution). Personal history (last 50 runs `{date, score, bestMs}`) stored locally.

4. Screens & UI

State	Elements
`menu`	Title, one-line rule, best score + best ms, ▶️ Start, mute toggle
`waiting`	Full-dark screen, "wait for it…" hint, HUD (hearts, score, round)
`signal`	Full-bleed color, optional overlay word (Stroop), HUD persists
`feedback`	Center overlay text + tone color, 400 ms
`gameover`	Card: score, best ms, rounds, top-% badge, 📤 Share, ↻ Retry, ⌂ Menu
`paused`	Overlay "Paused — tap to resume" (auto on `visibilitychange`)

Whole screen is the tap target (`pointerdown`); `touch-action:none` on the play area;
Space/Enter equivalent on desktop; HUD height fixed across states (no relayout).

5. Data model (localStorage, guarded wrapper)

```
game:tap-or-dont-tap:best     → { score: 1240, bestMs: 187 }
game:tap-or-dont-tap:history  → [{ date: '2026-09-10', score: 1240, bestMs: 187 }, …max 50]
game:tap-or-dont-tap:muted    → true|false
```

6. File structure & function inventory

```
tap-or-dont-tap/
  index.html    # state sections + gameover card
  style.css     # flash colors, HUD, overlays
  game.js       # constants → pure core → state machine → DOM wiring
```

Pure core (no DOM — the test surface, currently in `game.js`, mirrored in the site test file): `windowMs(round)`, `pointsForGreen(streak)`, `pickSignal(round, rng)`, `resolveTap({signalKind, expectsTap, tapped, elapsedMs})`, `percentileFor(score)`.
DOM layer: `startRun`, `beginRound`, `showSignal`, `handleTap`, `expireSignal`, `finishRound`, `loseHeart`, `gameOver`, `pauseRun`, `showFeedback`, audio helpers.

7. Edge cases (verify each once)

1. Hidden tab during WAITING → pause; on return restart the wait from 0. Hidden during SIGNAL → counts as a miss (anti-cheat).
2. Tap within 50 ms of a state change → single tap only (pointerdown debounce).
3. Private mode / storage disabled → game runs, persistence silently skipped.
4. Phone sleep mid-wait → `visibilitychange` → pause path.
5. Stroop × swap overlap (rounds 21–23) → swap inverts green/red; decoys never tappable.
6. Rapid restart from gameover → all pending timers cleared.
7. Reaction ≤ 0 ms or > window + 50 ms → discarded as anomaly (clock sanity).

8. Testing plan

Existing: `apps/frontend/src/__tests__/games-tap-or-dont-tap.test.ts` — keep green.
Required cases (verify present; add missing): `windowMs` ramp incl. floor; `pointsForGreen` cap; `resolveTap` truth table (5 signal kinds × tap/late/no-tap); seeded `pickSignal` sequences per band; `percentileFor` monotonic + bounds.

9. Task breakdown

P0 — playable core — DONE (2026-09-09 build pass)

- ☑ State machine + round loop; signal kinds; hearts; reaction capture; ramp; scoring
- ☑ HUD + gameover card; pause/resume (`visibilitychange` incl. wait-restart)

P1 — full rules & feel — DONE

- ☑ Streak multiplier; decoys; Stroop traps; rule swap; false starts
- ☑ Feedback overlays; WebAudio + mute; baked percentile badge

P2 — persistence/share/QA

- [x] Best score + history persistence; share chain
- [ ] **Delete `analyticsCandidates` / `trackGamePlayed` stubs** (isolation decision — they must never post to `/api/*`)
- [ ] Optional: migrate inline storage/audio helpers to `../shared/`
- [ ] Optional: extract pure core to `core.js`; keep site test file green
- [ ] QA gate (master README §5): offline play + isolation greps

P3 — polish

- [ ] Reduced-motion variant of flash; tuning pass after 5 test sessions
- [ ] History sparkline on menu; haptics on heart loss; sound design pass

10. Acceptance criteria

- §7 edge cases each verified once (note date when done); §8 tests green.
- 20 consecutive rounds, no stuck state; restart from gameover < 300 ms to first signal.
- Share works through all three fallback paths (share sheet / clipboard / prompt).

11. Deferred coupling (intentionally NOT built)

Footer/nav links, `game_played` analytics posting, achievements, leaderboard backend — per master README §7. The analytics stubs in `game.js` are to be **deleted**, not wired.

12. Remaining work (from code review 2026-09-10)

1. Remove analytics stubs (P2 — the only required code change).
2. Optional shared-utils migration + `core.js` extraction (P2, non-blocking).
3. Reduced-motion + tuning (P3).

2D GAMES · SAMPLE BUILD PLAN

Game 02 — Tic Tac Toe (Complete Plan)

Game 2 of 7 Complexity: ⭐ Low Estimate: 1 day Status: not started

> Complete plan (supersedes the 2026-09-09 sample). Status: **built** — P0–P2 complete plus misère from P3; X/O theme picker skipped. `apps/frontend/public/games/tic-tac-toe/` (`index.html`, `style.css`, ESM `game.js`, `game.test.html`); jest twin at `src/__tests__/games-tic-tac-toe.test.ts` incl. the exhaustive minimax sweep. Constants below are the spec of record; §9–§11 list the only remaining work.

1. Overview

Classic tic tac toe: two players on one device, or single player vs. the computer at three AI levels. Hook is the running series score ("first to 5?") and a provably unbeatable Hard AI.

2. Complete game spec

- 3×3 board; **X moves first in round 1**; alternate marks each round; the **loser of a round starts the next**; after a draw, the alternating starter flips.
- Win = 3 in a row (8 lines: 3 rows, 3 cols, 2 diagonals). Draw = full board, no win.
- **Modes:** `2P local` (pass-and-play) · `1P` vs computer with levels:
  - **Easy** — uniformly random legal move.
  - **Medium** — win/block heuristic:
    1. take a winning move if one exists
    2. block an opponent win
    3. otherwise random legal move
  - **Hard** — full minimax (unbeatable); ties broken by first-found move in scan order (deterministic for tests).
- **Misère variant** (toggle): completing 3-in-a-row **loses**. Win detection is the same `checkWinner`; the *interpretation* flips (winner of a line loses the round). Draw rules unchanged.
- Illegal input impossible: occupied cell, finished round, or out-of-turn tap are no-ops; input locked during AI "thinking" tick (~300 ms artificial delay for feel).

3. Series model

Series = per-mode running tally. Persisted per `(mode, level)` key; Reset zeroes the current series only.

4. Screens & UI

State	Elements
`menu`	Mode pick (1P/2P), level pick (1P), misère toggle, series scoreboard, ▶️ Play
`playing`	Board grid, turn indicator (mark + color), series mini-score, ⌂ Menu
`roundEnd`	Overlay: "X wins! 🎉" / "O wins!" / "Draw 🤝" (misère wording flips), winning line
	highlighted through the 3 marks, [Next round] [Menu]

Board = CSS grid; marks render as inline SVG strokes with draw-in animation. Winning line gets a distinct highlight class. Fits 360 px viewport with no scroll.

5. Data model (localStorage)

```
game:tic-tac-toe:series:<modeKey>  → { x: 3, o: 2, d: 1 }   # modeKey = '2p' | '1p-easy' | '1p-medium' | '1p-hard'
game:tic-tac-toe:prefs             → { mode, level, misere, muted }
game:tic-tac-toe:best              → n/a (no score; share uses the series line)
```

6. File structure & function inventory

```
tic-tac-toe/
  index.html       # menu / playing / roundEnd sections
  style.css        # grid, marks, win highlight, overlays
  game.js          # ESM: pure logic exported + DOM wiring
  game.test.html   # browser twin of the jest suite
```

Pure exported logic: `newBoard()`, `applyMove(board, i, mark)`, `checkWinner(board) → { winner, line } | 'draw' | null`, `legalMoves(board)`, `minimaxScore(board, mark)`, `bestMove(board, mark)` (hard), `mediumMove(board, mark)`, `easyMove(board, mark, rng)`, `resolveMisere(result)`. DOM layer: screens, `renderBoard/renderCell`, `highlightWin(line)`, `isAiTurnNow()`, series storage, share.

7. Edge cases

1. occupied / finished / out-of-turn taps = no-ops; double-tap debounce 50 ms.
2. AI never acts during `roundEnd`; Next-round resets board but not series.
3. Misère + AI: Hard minimax optimizes the *misère* objective (inverted leaf scores) — covered by the exhaustive test at this variant too.
4. Storage disabled → defaults, no persistence, no crash.
5. Series key changes when mode/level changes (no cross-mode pollution).
6. Draw starter alternation persists only within a session (not in storage) — stated.

8. Testing plan (jest twin + game.test.html)

- `checkWinner`: all 8 lines, draw, empty, mid-game null.
- Hard AI: exhaustive sweep — never loses from any reachable position (both normal and misère objectives).
- Medium: always takes an available win; always blocks an immediate threat.
- Series persistence round-trip; mode-key isolation.
- Misère interpretation flips round outcome without changing `checkWinner`.

9. Task breakdown

P0/P1/P2 — DONE (2026-09-09/10 build pass)

- [x] 2P local, win/draw detection, win-line highlight, round flow, series scoreboard
- [x] 1P Easy/Medium/Hard (minimax), misère variant, SVG marks + animations
- [x] Persistence, tests (incl. exhaustive unbeatable sweep), game.test.html

P2 — remaining

- [ ] **Resolve analytics coupling** — `game.js` (~lines 531–582) POSTs `game_played` with `?api=` base resolution. Under isolation: delete this block (owner call — see master README §7 conflict note).
- [ ] QA gate (master README §5): offline play + isolation greps.

P3 — remaining polish

- [x] Misère variant — done
- [x] Board flip animation — done
- [ ] X/O theme picker (skipped by owner decision — keep skipped unless requested)

10. Acceptance criteria

- §7 cases verified; §8 tests green in jest and browser twin.
- Hard AI unbeatable in exhaustive sweep (both objectives).
- Series survives reload; Reset works; 360 px fits with no scroll.

11. Deferred coupling (intentionally NOT built)

Online multiplayer (needs backend rooms), footer/nav links, Play Hub card, achievements — per master README §7. Existing analytics POST block is the one live violation; flagged above, not silently removed.

2D GAMES · SAMPLE BUILD PLAN

Game 03 — Sliding Puzzle (Complete Plan)

Game 3 of 7 Complexity: ⭐ ⭐ Medium Estimate: 1–2 days Status: not started

> Complete plan (supersedes the 2026-09-09 sample). Status: **not started** — build-ready as written. Slug `sliding-puzzle`; folder `apps/frontend/public/games/sliding-puzzle/`.

1. Overview

Classic 15-puzzle: slide tiles into order. Hook is the personal record pair — "4×4 in 2:41 · 213 moves" — plus guaranteed-solvable shuffles so nobody ever hits an impossible board.

2. Complete game spec

- Sizes: **3×3** (default), **4×4** (classic), 5×5 (P3). Tiles numbered 1..N^2-1, blank last.
- **Shuffle:** from the solved state, apply random legal moves — 120 (3×3), 250 (4×4), 400 (5×5) — never undoing the immediately previous move. Solvability is guaranteed by construction; the inversion-count check runs anyway in tests.
- **Move counting:** a single-tile slide = 1 move; a row/column segment push = 1 move per tile displaced.
- **Timer:** starts on the first move, stops on the winning move; accumulates across pauses (`visibilitychange`), never runs during `menu` / `won`.
- **Win:** tiles in order, blank at end → overlay (§4). Input locked after the winning move.
- **Score:** `score = max(0, sizeBonus - floor(moves/2) - floor(seconds/5))` with `sizeBonus` 300 / 500 / 800 for 3×3 / 4×4 / 5×5.
- **Wrong-tile feedback:** tapping a tile not in the blank's row/column = shake animation, no move, no move-count change.

3. Progression model

Best records per size (time + moves). Progression is self-referential (beat your record); "won at size N" unlocks nothing (all sizes always selectable — no fake gating).

4. Screens & UI

State	Elements
`menu`	Size picker (3 cards with best time/moves per size), ▶️ Play, mute toggle
`playing`	Board (CSS grid), HUD: moves, timer, [Restart] [Shuffle] [⌂ Menu]
`paused`	Overlay on `visibilitychange` / ⏸️ — board hidden (no sneaking moves), timer held
`won`	Overlay: 🎉 time, moves, score, per-size best badges ("NEW BEST!"), [Play again] [Bigger grid] [📤 Share]

Tiles are absolutely-positioned buttons moved via `transform: translate` transitions (180 ms ease); the blank is a missing tile. Board sizing: `min(92vw, 60dvh)` square.

5. Data model (localStorage)

```
game:sliding-puzzle:best:<size>  → { timeMs: 161000, moves: 213 }     # size = 3|4|5
game:sliding-puzzle:prefs        → { size: 3, muted: false }
```

6. File structure & function inventory

```
sliding-puzzle/
  index.html
  style.css
  core.js     # pure model — the test surface
  game.js     # rendering, input, timer, audio, share
```

`core.js` exports: `solvedBoard(n)`, `neighbors(i, n)`, `legalMoves(board, n)`, `shuffle(n, movesCount, rng) → board` (no-undo rule), `slideTile(board, n, index) → { board, moved } | null` `(single tile or segment),` `isSolved(board)` `,` `scoreFor(size, moves, seconds)` `.` `game.js`: render from board array (keyed by tile value), pointer + keyboard input, timer accumulator, best-score logic, share text.

7. Edge cases

1. Segment slide: tap tile k in blank's row/col → tiles between k and blank shift one step toward blank; move count += displaced count; animation staggered 40 ms/tile.
2. Shuffle never leaves the board solved (assert; re-shuffle if it somehow lands solved).
3. Pause hides the board (overlay is opaque) — no solving while paused; timer freezes.
4. Keyboard focus: arrows move the tile *into* the blank (tile travels in arrow direction); focus ring visible; Enter = same as tap on focused tile.
5. Storage disabled → no records, game still fully playable.
6. Rapid tap during transition animation → state updates immediately, animation catches up (never queue clicks).
7. `won` input lock: any tap/slide after the final move is a no-op until an overlay button is used.

8. Testing plan (core.js under node:test or test.html)

- 100 shuffles per size → `isSolved` false + inversion-parity solvable (3×3 & 4×4 rules).
- No-undo rule: replay any shuffle log → no move is the inverse of its predecessor.
- `slideTile`: adjacent slide; row segment (2 and 3 tiles); column segment; illegal tile → null; move counts correct.
- `isSolved` on solved + known-shuffled boards.

- `scoreFor`: clamps at 0; monotonic in moves and seconds.

9. Task breakdown

P0 — playable core

- [ ] `core.js` model: solved/neighbors/legalMoves/slideTile/isSolved/shuffle + unit tests
- [ ] Board rendering with transform transitions; tap + segment slide; move counter
- [ ] Timer (accumulate across pause); win detection + overlay; restart/shuffle buttons

P1 — full rules & feel

- [ ] 3×3/4×4 size switch with per-size records; pause overlay (board hidden)
- [ ] Slide/snap animations, staggered segment push, wrong-tile shake, sound blips + mute
- [ ] Keyboard play (arrows + Enter) with visible focus

P2 — persistence/share/QA

- [ ] Best records per size + "NEW BEST" badge; share text (`{size}×{size} in m:ss · moves`)
- [ ] QA gate (master README §5): offline, isolation greps, 10-min phone session

P3 — polish

- [ ] 5×5 size; picture mode (CSS background-position split of any bundled-free image); single-level undo; solve-demo animation; swipe input

10. Acceptance criteria

- §8 tests green; shuffles always solvable (automated proof, 100 runs/size).
- Timer exact across pause/resume (±50 ms vs wall clock of played time).
- No input accepted after win; 3×3 fits 360 px with HUD visible, no scroll.
- Restart ≤ 1 tap; shuffle produces a visibly mixed board every time.

11. Deferred coupling (intentionally NOT built)

Footer/nav links, analytics events, achievements, backend leaderboards — per master README §7.

2D GAMES · SAMPLE BUILD PLAN

Game 04 — Word Puzzle (Complete Plan)

Game 4 of 7 Complexity: ⭐ ⭐ Medium Estimate: 2–3 days Status: not started

> Complete plan (supersedes the 2026-09-09 sample). Status: **not started** — build-ready. Interpretation decided: **word search** (master README §8 #1–2). Slug `word-puzzle`; folder `apps/frontend/public/games/word-puzzle/`.

1. Overview

Themed word search: drag a line through letters to find hidden words; correct finds lock in with colored highlights, wrong picks flash red. Hook: 3-star levels and the daily seed (P3) that gives everyone the same puzzle.

2. Complete game spec

- **Content:** 4 themes (Animals 🦁, Food 🍎, Tech 💻, Space 🚀) × 3 levels, shipped as static `data/themes.json` (schema §5). English, lowercase, 4–9 letters, no spaces, no duplicate words within a level.
- **Grid tiers:** L1 = 8×8, 5 words, directions {→, ↓}; L2 = 10×10, 7 words, + {↘, ↗}; L3 = 12×12, 10 words, all 8 directions incl. reversed.
- **Selection:** pointerdown on a cell anchors; pointermove samples cells via `document.elementFromPoint`, snapped to the grid; a valid drag follows one of the 8 direction vectors in a straight line. Live highlight while dragging; on release:
  - Letters match an unfound word (or its reverse where reversed dirs are on) → **locked** in the word's color, struck off the list, +pop animation. ✅
  - Otherwise → red flash 300 ms, **wrong picks +1**, selection clears. ❌
- **Hints:** 3 per level; a hint rings the first letter of a random unfound word.
- **Stars:** ⭐ finish the level · ⭐ time ≤ par · ⭐ `hintsUsed === 0 && wrongPicks ≤ 2`.
- **Par times:** L1 90 s · L2 150 s · L3 240 s (per level data, tunable).
- **Level flow:** complete → result overlay → next level → theme complete → badge. Progress persisted per level (§5).

3. Generator (pure, deterministic)

`generateLevel(levelData, seed)` using `mulberry32(seed)`:

1. Sort words longest-first. For each word: try up to 200 random placements (random direction from the tier's set, random start cell) — legal if in bounds and every cell is empty or holds the same letter (shared-letter crossings allowed).
2. If a word fails 200 attempts → restart the whole level (max 20 restarts, then throw — data bug, must never ship).

3. Fill empty cells from a letter bag: 60 % weighted toward the level's own letters, 40 % uniform A–Z (keeps grids solvable-feeling and dense).
4. Return `{ grid, placements: [{word, row, col, dir}], seed }`.

4. Screens & UI

State	Elements
`menu`	Theme cards (emoji, name, stars x/9, badge when complete), Continue banner, mute toggle
`level`	Letter grid + word list (chips; found = struck, colored), HUD: timer, hint stars, [Hint] [⌂]
`result`	Overlay: time, stars, words found, [Next level] [Replay] [📤 Share]; theme-complete variant

Word list sits below the grid at phone width (side-by-side ≥ 768 px). Grid cells sized `min(92vw, 100% / size)`; touch highlight uses per-word colors from a fixed 10-color accessible palette (4.5:1 on white).

5. Data model

```
data/themes.json:
{ "themes": [ { "id": "animals", "name": "Animals", "emoji": "🦁",
    "levels": [ { "size": 8, "parSec": 90, "words": ["LION","ZEBRA", …] }, … ] }, … ] }

localStorage:
game:word-puzzle:progress  → { "<themeId>:<lvl>": { stars: 0-3, bestTimeMs: 84200 } }
game:word-puzzle:prefs     → { muted: false }
```

6. File structure & function inventory

```
word-puzzle/
  index.html
  style.css        # grid, highlight pills, flash animations
  core.js          # generator + validation (pure, seeded) — test surface
  game.js          # input, rendering, timer, stars, persistence
  data/themes.json
```

`core.js` exports: `mulberry32(seed)`, `generateLevel(level, seed)`, `lineCells(grid, start, end) → cells | null` `(8-direction check)`, `lettersAt(grid, cells) → string`, `matchesWord(letters, word, allowReverse)`, `starsFor(parSec, timeSec, hintsUsed, wrongPicks)`. `game.js`: pointer/tap-tap/keyboard input, highlight rendering, timer, progress, share.

7. Edge cases

1. Diagonal reverse drags (e.g. ↙ when reversed on) must validate as the word reversed.
2. Drag that leaves the grid → clamp to last valid cell; release outside → evaluate.
3. Re-finding a found word → counts as correct-but-noop (no star penalty, no duplicate).
4. Wrong selection on cells that partially spell a word (prefix) → still wrong (full word required) — documented in-game via the red flash.
5. Tap-tap mode: first tap anchors (cell pulses), second tap submits; tapping the anchor again cancels.
6. Keyboard: arrows move focus; Enter anchors/submits; grid is one tab stop (roving focus), `aria-label` per cell = "row r column c, letter X"; word chips labeled "word k, N letters" (never the word itself).
7. `visibilitychange` pauses the timer; level restarts only via user action.
8. Storage disabled → progress not saved, everything else works.

8. Testing plan (core.js)

- Seeded determinism: same seed → identical grid/placements (snapshot).
- Every word present in `placements`, within the tier's allowed directions.
- Generated grid: all placements verified by `lineCells` + `lettersAt` round-trip.
- Generator termination: 100 seeds per shipped level, zero throws.
- `lineCells`: 8 directions, out-of-bounds, non-collinear → null.
- `matchesWord`: exact, reverse (allowed vs not), case handling.
- `starsFor`: boundary table (par ±1 s, hints 0/1, wrong 2/3).

9. Task breakdown

P0 — playable core

- [ ] `core.js`: mulberry32, generateLevel, lineCells, lettersAt, matchesWord + tests
- [ ] Grid rendering + drag selection with live highlight and correct/wrong resolution
- [ ] Word list check-off, level timer, win detection, next-level flow

P1 — full rules & feel

- [ ] 4 themes × 3 levels data (`themes.json`) + data lint (lengths, dupes, charset)
- [ ] Reversed directions at L3; hints; star rating; color pills; red flash; pop anim
- [ ] Progress persistence + Continue; sound blips + mute

P2 — persistence/share/QA

- [ ] Tap-tap + full keyboard path; a11y labels (§7.5–7.6)
- [ ] Share text (`{words} words in m:ss · ⭐⭐⭐`); QA gate (master README §5)

P3 — polish

- [ ] Daily puzzle (seed = `YYYYMMDD` hash → same grid for everyone, share-friendly); confetti on theme completion; more themes

10. Acceptance criteria

- §8 tests green; every shipped level solvable and every word placeable (generator proof).
- Wrong picks never permanently mark letters; found letters stay locked.
- Drag usable at 360 px without zoom; grid + chips visible during a drag.
- Seeded daily mode reproduces identical grids across reloads.

11. Deferred coupling (intentionally NOT built)

Riddle/content-pipeline integration (backend word lists), footer/nav links, analytics events, achievements — per master README §7.

2D GAMES · SAMPLE BUILD PLAN

Game 05 — Continuous Runner / Hurdles (Complete Plan)

Game 5 of 7 Complexity: ⭐ ⭐ Medium Estimate: 2–3 days Status: not started

> Complete plan (supersedes the 2026-09-09 sample). Status: **not started** — build-ready.
> Slug `hurdle-runner`; folder `apps/frontend/public/games/hurdle-runner/`.
> Its engine is the foundation for Spirit Runner (07) — build to the structure in §6.

1. Overview

Endless runner: the world scrolls, hurdles approach, you jump. One input, instant restart, meters as the bragging number. Success metric: median first session ≥ 3 runs; death always reads as the player's fault.

2. Complete game spec

World & camera

- Virtual viewport 900×500 px logical, scaled to fit (letterboxed, `devicePixelRatio` aware).
- Ground line at y=420. Parallax: far hills ×0.2, near trees ×0.5, ground ×1 of scroll speed.
- Player fixed at x=225 (25 %).

Physics (fixed timestep 1/120 s, accumulator, 250 ms frame clamp)

Constant	Value	Notes
`SCROLL_START`	320 px/s	+6 px/s per second, cap 900
`JUMP_VY`	−820 px/s	on jump
`GRAVITY`	2200 px/s²	
`JUMP_CUT`	0.55	releasing early caps upward velocity at 55 %
`COYOTE_MS`	80	grace after leaving ground
`BUFFER_MS`	100	early tap counts when landing
`HITBOX_INSET`	15 %	forgiveness on the player AABB

Obstacles (spawned ahead at x=950, culled behind x=−100)

Type	Size (w×h)	Introduced	Notes
Hurdle	24×40–60 px	0 m	low jump
Tall barrier	28×90 px	500 m	early, full-height jump

Type	Size (w×h)	Introduced	Notes
Double hurdle	two hurdles 140–180 px apart	1500 m	forces full-speed chained jumps

- **Spawner fairness:** gap between obstacles sampled 380–700 px, but never below `minGap(speed) = speed × 1.05 + 120` px (a full jump always fits). Doubles count the inner spacing toward the next gap too.
- **Collision:** AABB, player box ×0.85 inset; obstacle boxes full. Death = game over (one hit — decision master README §8 #9).
- **Score:** meters = px/10 (distance), +10 per obstacle cleared (its right edge passes player x). 💚 pickup (P2): once per run, spawns 600–900 m; grants one extra life → on fatal hit, consume heart + 1 s invulnerability flash.

Tiers (HUD announces "Speed up!")

- Tier 1: 0–500 m (hurdles only) · Tier 2: 500–1500 m (+tall) · Tier 3: 1500 m+ (+doubles).

3. Screens & UI

State	Elements
`menu`	Title, best distance, control hint, ▶️ Run, mute toggle
`playing`	Canvas world; HUD: distance, next-tier progress bar, hearts (only if 💚 held)
`paused`	Overlay (auto on `visibilitychange` / ⏸️)
`gameover`	Distance, obstacles cleared, best badge ("NEW BEST!"), [↻ Retry] [📤 Share]

`?debug=1` query flag draws hitboxes and spawner-gap markers (dev aid, documented).

4. Controls

- **Jump:** tap anywhere / Space / ↑. `pointerdown`; hold = higher (jump cut on release).
- Coyote + buffering exactly as specced (feel-critical, in `Player.update`).
- Nothing else at MVP — one primary action. Down/fast-fall is P3.

5. Data model (localStorage)

```
game:hurdle-runner:best   → { distanceM: 1204 }
game:hurdle-runner:prefs  → { muted: false }
```

6. File structure & function inventory (engine is 07's base)

```
hurdle-runner/
  index.html
  style.css
  core.js     # pure: physics step, spawner, collision, scoring — test surface
  engine.js   # loop/camera/parallax/input (generic — 07 forks this file)
  render.js   # draw calls: world, player, obstacles, HUD (procedural art)
  main.js     # state machine + glue
```

`core.js` exports: `createPlayer()`, `stepPlayer(player, dt, input)` (jump/buffer/coyote/gravity), `createSpawner(rng)`, `nextSpawn(spawner, speed, lastX)` (fairness rule), `aabbHit(playerBox, obsBox, inset)`, `meters(px)`, `tierFor(m)`. `engine.js`: `createLoop` (from `shared/loop.js`), camera/parallax scroll, `visibilitychange` wiring. `render.js`: pure draw functions taking state (no logic). `main.js`: `menu/playing/paused/gameover` transitions.

7. Edge cases

1. Backgrounded mid-air → pause freezes physics; resume continues the same arc exactly.
2. 60 Hz vs 120 Hz displays → identical trajectories (fixed step; render reads sim state).
3. Frame spike > 250 ms → clamped (no tunneling through obstacles).
4. Jump pressed during gameover/menu → ignored (input only in `playing`).
5. 💚 held + fatal hit → heart consumed, invulnerability ignores hits for exactly 1 s (no double-consume), HUD heart count updates.
6. Storage disabled → no best score, playable.
7. Tab-crash recovery: no mid-run persistence (stated; run is lost — acceptable).

8. Testing plan (core.js, node:test / test.html)

- Jump arc: from ground, max jump clears a 90 px tall barrier at min speed 320 px/s.
- Jump-cut: release at 50 % rise → apex lower than full hold (assert heights).
- Spawner sweep: for speed 320→900 step 20, 500 spawns each — no gap < `minGap(speed)`; doubles' inner spacing within 140–180 px.
- Collision: inset math (corner cases: 1 px overlap inside inset = no hit; at box edge = hit).
- Scoring: meters conversion; +10 awarded exactly once per obstacle.

9. Task breakdown

P0 — playable core

- [ ] `core.js` physics + spawner + collision + scoring (+ tests §8)
- [ ] `engine.js` loop, ground scroll, camera; player jump with coyote/buffer/cut

- [ ] Hurdles, speed ramp, tiers, game over; menu/gameover states; best distance

P1 — full rules & feel

- [ ] Parallax layers; procedural player run/jump animation (shapes); landing dust
- [ ] Tall barrier + double hurdles; tier announcements; WebAudio jump/land/crash + mute
- [ ] Pause overlay + `visibilitychange` (shared loop handles)

P2 — persistence/share/QA

- [ ] 💚 pickup; share (`Ran {m} m — beat that!`); `?debug=1` hitboxes
- [ ] QA gate (master README §5): 60 fps phone check, offline, isolation greps

P3 — polish

- [ ] Day/night palette shift per 500 m; slide/duck obstacle; fast-fall; gamepad

10. Acceptance criteria

- §8 tests green; no impossible spawns (automated sweep proof).
- Identical jump arcs at 60/120 Hz (sim log comparison).
- Death never ambiguous: `?debug=1` boxes match shipped hitbox math exactly.
- Restart ≤ 1 tap, < 300 ms to first obstacle; 60 fps on a mid-range phone with particles on.

11. Deferred coupling (intentionally NOT built)

Footer/nav links, analytics events, achievements, leaderboards — per master README §7.

Source: plan/games markdown · generated 2026-09-09 · part of the 2D Games sample plans (plan/games/pdf).

2D GAMES · SAMPLE BUILD PLAN

Game 06 — Flying Snake / Flappy (Complete Plan)

Game 6 of 7 Complexity: ⭐ ⭐ Medium Estimate: 1–2 days Status: not started

> Complete plan (supersedes the 2026-09-09 sample). Status: **not started** — build-ready.
> Slug `flying-snake`; folder `apps/frontend/public/games/flying-snake/`.
> Branding decided: page title "Flying Snake" (master README §8 #8).

1. Overview

Flappy-style: tap to flap against gravity, thread the gaps. One-tap, instantly restartable, brutally scoreable. Success metric: median session ≥ 8 runs ("one more try" loop confirmed); death always reads as fair.

2. Complete game spec

Physics (fixed timestep 1/120 s, accumulator, 250 ms frame clamp)

Constant	Value	Notes
`GRAVITY`	1800 px/s²	
`FLAP_VY`	−480 px/s	one flap per press (no hold-to-fly)
`TERMINAL_VY`	+700 px/s	max fall speed
`SNAKE_X`	30 % width	fixed; world scrolls left
`SNAKE_R`	14 px	visual radius; hitbox radius ×0.9 (forgiving)
`WORLD_SPEED`	160 px/s	constant — difficulty scales via gap, not speed

Obstacles (pipe/vine pairs)

- **Gap:** 170 px, shrinks 2 px per point, floor 120 px.
- **Spacing:** 300 px horizontal, constant (density feels fairer than shrinking both axes).
- **Gap center:** random, ≥ 80 px + gap/2 from top and bottom edges.
- **Ceiling grace:** first 2 s of a run the ceiling clamps (no death); ground always lethal.
- **Collision:** circle (×0.9) vs pipe rects + ground strip; circle-rect standard clamp test.
- **Score:** +1 when the snake's x passes a pair's right edge (counted once per pair).
- **Medals:** bronze 10 · silver 20 · gold 40 · platinum 75 (gameover card).

Feel carriers

- **Tilt:** angle = `clamp(map(vy, −480…+700, −25°…+80°))` — the snake reads velocity.
- **Trailing segments:** 5 circles sampled from past positions every 40 ms with a small sinusoidal wiggle — sells "flying snake" with zero art assets.

- Death: white flash 120 ms + 300 ms screen shake + fall animation, then gameover card.

3. Screens & UI

State	Elements
`menu`	Title, best score + medal, bobbing snake animation, "Tap to start"
`ready`	World visible + frozen, "Tap to flap" hint; first tap starts physics mid-screen
`playing`	World + HUD score (large, top-center) + mute toggle
`paused`	Overlay (auto on `visibilitychange`)
`gameover`	Score, best, medal, [↻ Retry] [📤 Share]

Virtual viewport 720×960 logical (portrait), scaled/letterboxed, `devicePixelRatio` aware. `?debug=1` draws hitbox circle + pipe rects (dev aid).

4. Controls

Tap anywhere / Space / ↑ = flap. `pointerdown` based; multi-touch extra flaps within 50 ms ignored; `touch-action:none`. No other input at MVP.

5. Data model (localStorage)

```
game:flying-snake:best   → { score: 23 }
game:flying-snake:prefs  → { muted: false }
```

6. File structure & function inventory

```
flying-snake/
  index.html
  style.css
  core.js     # pure: physics step, spawner, collision, scoring — test surface
  render.js   # procedural draw: snake segments, vines, ground, HUD
  main.js     # state machine (menu/ready/playing/paused/gameover) + glue
```

`core.js` exports: `createSnake()`, `stepSnake(snake, dt, flapped)` (gravity/terminal/flap), `createSpawner(rng)`, `nextGap(spawner, score)` (margins + shrink rule), `circleRectHit(cx, cy, r, rect)`, `tiltFor(vy)`, `medalFor(score)`. `main.js` owns the run lifecycle: `ready` starts frozen; first flap starts physics; score-on-pass check in the update loop.

7. Edge cases

1. `ready` state: physics frozen, snake bobs at safe y (never inside a spawn line); first flap both starts the run and applies the flap.
2. Backgrounded → pause via shared loop; resume continues the exact fall state.
3. Flap during `gameover` → ignored; retry is a deliberate button (prevents skip-death).
4. Gap center RNG can't place a gap within margins (assert in spawner tests).
5. Score increments once per pair even if the snake weaves back (x only moves forward — snake x is fixed; world moves — so structurally impossible, test asserts it).
6. Storage disabled → no best/medal, playable.
7. Tab-titling: document.title swap on death is P3 fluff — skip.

8. Testing plan (core.js)

- Flap arc: from gap center y, a single flap clears the smallest gap (120 px) without a second flap when centered; two flaps suffice from gap edges (assert math).
- Spawner sweep: 1,000 gaps across scores 0–80 → all margins ≥ 80 px + gap/2; gap ≥ 120.
- `circleRectHit`: corner cases (corner clamp), 1 px inside/outside inset behavior.
- `tiltFor`: endpoints (−25° at min vy, +80° at terminal), monotonic.
- `medalFor`: 9/10/19/20/39/40/74/75 boundaries.

9. Task breakdown

P0 — playable core

- [ ] `core.js` physics + spawner + circle-rect collision + score-once (+ tests §8)
- [ ] World rendering: pipes, ground, snake body with tilt + trailing segments
- [ ] menu/ready/playing/gameover states; ceiling grace; death flash/shake; instant retry
- [ ] Best score persistence; pause via `visibilitychange`

P1 — full rules & feel

- [ ] Gap shrink ramp; medals on gameover card; score pop animation
- [ ] WebAudio flap/score/hit + mute; menu bobbing animation

P2 — persistence/share/QA

- [ ] Share (`Flew through {n} gaps — beat that!`); `?debug=1` hitboxes
- [ ] QA gate (master README §5): fps check, offline, isolation greps

P3 — polish

- [ ] Moving pipes (gentle bob) after score 50; night palette every 25 points; ghost of best run flying alongside

10. Acceptance criteria

- §8 tests green; 1,000-gap sweep proof stored in the test file.
- Deterministic physics across 60/144 Hz (sim log comparison).
- First flap from `ready` never spawns the snake into a pipe.
- Restart ≤ 1 tap, < 300 ms to playable; 60 fps mid-range phone.

11. Deferred coupling (intentionally NOT built)

Footer/nav links, analytics events, achievements, leaderboards — per master README §7.

2D GAMES · SAMPLE BUILD PLAN

Game 07 — Spirit Runner (Complete Plan)

Game 7 of 7 Complexity: ⭐ ⭐ ⭐ High Estimate: 5–7 days Status: not started

> Complete plan (supersedes the 2026-09-09 sample). Status: **not started** — build-ready, **requires Game 05's engine first** (forks `engine.js`). Slug `spirit-runner`; folder `apps/frontend/public/games/spirit-runner/`. Highest complexity (5–7 days); phased A–D so every phase ends shippable.

1. Overview

Mystical-forest endless runner with light puzzle choices: run, dodge, collect spirit orbs for powers, and at path splits pick the right **rune gate** — wrong pick banishes you to the riskier, richer **shadow realm**. Depth from choices, not controls (one thumb).

2. Complete game spec

Phase A — core run (fork of Game 05)

- Jump (as 05: coyote/buffer/cut) **plus slide**: swipe down / ↓ / S — 600 ms, hitbox drops to 40 % height; slide-jump cancels into a hop.
- Obstacles:

Type	Clearance	Look (procedural)	Introduced
Fallen log	jump	brown rounded rect	0 m
Low branch	slide	green bar at head height	300 m
Spirit guardian (tall)	jump only	glowing wisp column	800 m
Spirit guardian (low)	slide only	hovering wisp disc	800 m
Rune trap	time it	floor spikes pulsing: 1.2 s cycle — 0.7 s dark (safe), 0.5 s lit (lethal)	1200 m

- Speed ramp as 05 (320→900 px/s). **Hearts: 2**; hit = −1 heart + 1.2 s invulnerability (blink); 0 → game over (deliberately softer than 05 — longer-form run).
- Palette shifts by depth tier (dawn → dusk → night forest).

Phase B — orbs & powers

- Orbs spawn in arcs/lines riding the spawner (arc of 5–7 over a jump, line of 4 on flat). Meter: 10 orbs → full. Power = next in a **fixed cycle** (readable): Double jump → Dash → Slow time (durations in the table below).

Power	Duration	Effect
Double jump	8 s	second mid-air jump
Dash	1.5 s	invulnerable, destroys the next obstacle hit (exactly one)

Power	Duration	Effect
Slow time	4 s	world ×0.6 (audio pitch drops too)

- Power button (bottom-right, thumb reach) lights when meter full; keyboard E; optional auto-trigger setting.

Phase C — rune gates & shadow realm (the twist)

- Every **600 m ± 100** the path splits into two gates; a hint banner shows 2 s before. `gates.js` (pure) defines **5 rules**:

Rule	Hint text	Correct gate
Parity	"Even spirits walk the left path."	side matching `orbsSinceLastGate % 2`
Shape echo	"Follow the doubled symbol."	gate whose rune == the previous gate's rune
Negation	"The moon door lies."	NOT the gate matching the banner's shape hint
Sequence	"▲▲▲ …"	gate continuing the shown 3-symbol pattern
Color trail	"Trust the last light you caught."	side matching last orb's color

- API: `makeGate(ruleId, rng, runState) → { correctSide, hintText, runes }`; `resolveChoice(gate, side) → 'correct' | 'shadow'`. Rules cycle in order per run (learnable), order shuffles per run via seed.
- **Correct** → +100 points, continue forest. **Wrong** → **shadow realm**: 45 s, darker palette, obstacle density ×1.5, orbs worth ×2, guaranteed **+1 spirit shard** on survival; timer HUD countdown; auto-returns to forest (no soft-lock).

Phase D — characters & meta

- **Spirit shards:** `floor(meters/1000) + shadowSurvives + floor(correctGates/3)` per run.
- **Characters** (pick on menu; modifiers):

Character	Unlock	Modifier
Forest Spirit	start	power durations +50 %
Hunter	5 shards	every run starts with Dash charged
Monk	12 shards	Slow time also ×0.7 obstacle spawn rate

- Save persists shards + unlocks forever (best scores separate).

3. Screens & UI

State	Elements
`menu`	Title, character cards (locked show shard cost), best distance/shards, ▶️ Run, mute
`playing`	World + HUD: hearts, orbs → power meter + button, distance, depth tier
`gate`	In-world split + hint banner (run continues — no separate screen)

State	Elements
`shadow`	Palette swap + "SHADOW REALM" banner + shard countdown
`paused`	Overlay (auto on `visibilitychange`)
`gameover`	Distance, orbs, gates correct, shards earned, unlocks, [↻ Retry] [Character] [📤 Share]

Virtual viewport 900×500 like 05; `?debug=1` draws hitboxes + gate rule id (dev aid).

4. Controls

- Jump: tap / Space / ↑ (double jump while empowered). Slide: swipe down / ↓ / S.
- Power: button (bottom-right) / E; auto-trigger accessibility setting.
- Swipe-up = jump, swipe-down = slide (pointer gesture thresholds: 30 px, 120 ms).

5. Data model (localStorage)

```
game:spirit-runner:save → {
  bestDistanceM: 2340, shards: 17, unlocked: ['spirit'],
  character: 'spirit', settings: { autoPower: false, muted: false }
}
```

6. File structure & function inventory

```
spirit-runner/
  index.html
  style.css
  engine.js   # forked from 05: loop, camera, parallax, input (jump+slide+swipes)
  core.js     # pure: obstacles, orbs, powers, gates resolution, shards — test surface
  gates.js    # pure: the 5 rune rules + hint text
  render.js   # palettes (forest/shadow), glow (lighter composite), particles, HUD
  main.js     # state machine + meta (characters, unlocks)
```

`gates.js`: `RULES` array, `makeGate(ruleId, rng, runState)`, `resolveChoice(gate, side)`, `hintFor(ruleId, runState)`. `core.js`: `stepPlayer` (jump/slide states), spawner extensions (`spawnOrbArc`, `spawnTrap`, `spawnGuardian`), `applyPower(run, kind)`, `tickPowers(run, dt)` (timers incl. character modifiers), `shardsFor(run)`.

7. Edge cases

1. Gate while a power is active → power timers keep running through the split.
2. Wrong-gate during invulnerability → shadow realm still triggers (it's not damage).
3. Dash inside shadow realm destroys one obstacle only, as in forest.

4. Slow time stacks with Monk modifier → world ×0.6, spawn ×0.7 (multiplicative, capped once each — no double application).
5. Shadow realm timer always expires exactly (fixed-step accumulation; pause-safe); pays shards exactly once.
6. Unlocks persist across character switching; switching mid-menu only (never mid-run).
7. Rule sequence: 5 rules cycle; run seed shuffles the order; no rule repeats within a cycle; gates ≥ 600 m apart never overlap the shadow realm window (spawner defers).
8. Storage disabled → unlocks lost per session, playable.

8. Testing plan (gates.js + core.js)

- All 5 rules: `makeGate` + `resolveChoice` → correct side deterministic given a fixed `runState` (table tests per rule, 20 rng seeds each).
- Hint text present and never empty for any rule/state.
- Shard math: boundaries at 999/1000 m, 1/2/3 correct gates, shadow survive.
- Power timers: durations incl. Forest Spirit ×1.5; Dash single-destroy assertion.
- Shadow realm: density multiplier applied; timer expiry; single shard payout.
- Spawner: gate deferral rule (§7.7) over 1,000 simulated runs.

9. Task breakdown

Phase A — core run (≈2 days)

- [ ] Fork engine from 05; slide + low-branch/log/guardian/trap obstacles; trap pulse cycle
- [ ] 2 hearts + invulnerability; depth palettes; pause/blur; menu/gameover; best distance

Phase B — orbs & powers (≈1.5 days)

- [ ] Orb arcs in spawner; power meter + cycle; DJ/Dash/Slow implementations
- [ ] Power button + auto-trigger setting; WebAudio (collect/power/hit) + mute

Phase C — rune gates & shadow realm (≈2 days)

- [ ] `gates.js` 5 rules + hints (+ tests); split-path rendering; resolution flow
- [ ] Shadow realm: palette, density ×1.5, ×2 orbs, shard payout, timed return, banner

Phase D — characters & meta (≈1.5 days)

- [ ] Shard earning; unlock thresholds; 3 character modifiers; picker UI
- [ ] Balance pass: median first-run 30–45 s, each gate rule learnable ≤ 3 exposures
- [ ] Share (`Ran {m} m as {character}… {pts}`); QA gate (master README §5)

P3 — polish

- [ ] Sprite/hand-drawn art slot (post-MVP); music track; daily depth seed; extra rules
- [ ] Achievements/leaderboards: only if isolation decision is reversed (master §7)

10. Acceptance criteria

- Each phase ships playable alone (A = good runner; A+B = powered; +C = full twist; D = meta).
- §8 tests green — all gate rules proven deterministic per state.
- Shadow realm never soft-locks; pays exactly once; always exits.
- 60 fps mid-range phone with particles + glow on; pause never desyncs timers.
- One-thumb: jump, slide, power reachable at 360 px width.

11. Deferred coupling (intentionally NOT built)

Footer/nav links, analytics events, platform achievements integration, leaderboards —
per master README §7 (revisit only if the owner reverses isolation).

Source: plan/games markdown · generated 2026-09-09 · part of the 2D Games sample plans (plan/games/pdf).